0167. Two Sum II - Input Array Is Sorted

Category: Two Pointers | LeetCode: https://leetcode.com/problems/two-sum-ii-in

Problem

- Numbers is sorted ascending. Return 1-indexed positions [i, j] with numbers[i] + numbers[j] == target, using O(1) extra space.
- Example: [2,7,11,15], target 9 → [1,2].

Intuition

- Sorted order lets us steer: if the sum is too small, the only way to grow it is to move the left pointer right; if too big, move the right pointer left.
- Each move discards a whole row/column of impossible pairs, so we never miss the answer.

Brute Force

- All pairs O(n^2), or a HashMap like Two Sum (O(n) space, violates the constraint).

Optimal Approach

- L = 0, r = n-1.
- Sum = a[l] + a[r]. Equal → return [l+1, r+1]. Less → l++. Greater → r--.

Worked Example

```
numbers = [2, 3, 4], target = 6.

l=0(2) r=2(4) sum=6 -> return [1,3]
```

Java Solution

```java
public class TwoSumII {
    public int[] twoSum(int[] numbers, int target) {
        int l = 0, r = numbers.length - 1;
        while (l < r) {
            int sum = numbers[l] + numbers[r];
            if (sum == target) return new int[]{l + 1, r + 1};
            if (sum < target) l++;
            else r--;
        }
        return new int[0];
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Answer is 1-indexed.
- Use the sorted property; a HashMap is not accepted for the O(1) space requirement.